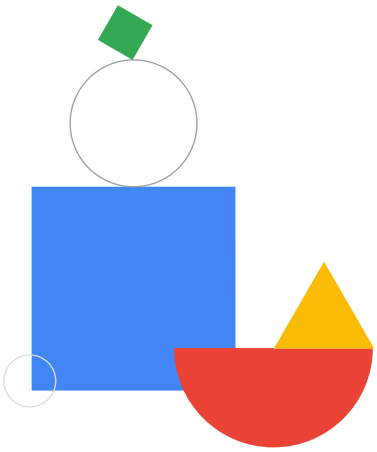
Employing Machine Learning Operations



Welcome to the first section of the Machine Learning Operations, or MLOps, Fundamentals course.

In this part of the course, you explore machine learning, or ML, models from an operational perspective.

First, you examine the challenges that ML practitioners currently face when they operationalize and make ML models available for production.

From there, you get an introduction to the concept of DevOps in ML.

Finally, you learn about the ML lifecycle.

Different roles throughout the ML lifecycle

Product manager	Data analyst	Data engineer	Data scientist	ML developer	ML engineer
Defines insights and objectives.	Queries and analyzes.	Gets clean, useful data.	Verifies that models work.	Creates intelligent apps.	Deploys models into production.

Before we start, let's look at what the “ML practitioner” term refers to.

Many users within an organization have a part to play in the ML lifecycle. For example, a product manager, who simply types a query to pull necessary insights from a data dashboard or data warehouse. Or a data scientist, who works on different aspects of building and validating models.

Think also about an ML engineer, who is responsible for the model to work without issues to serve end users in a production environment.

“ML practitioner” is used to describe all these different roles throughout the ML lifecycle.

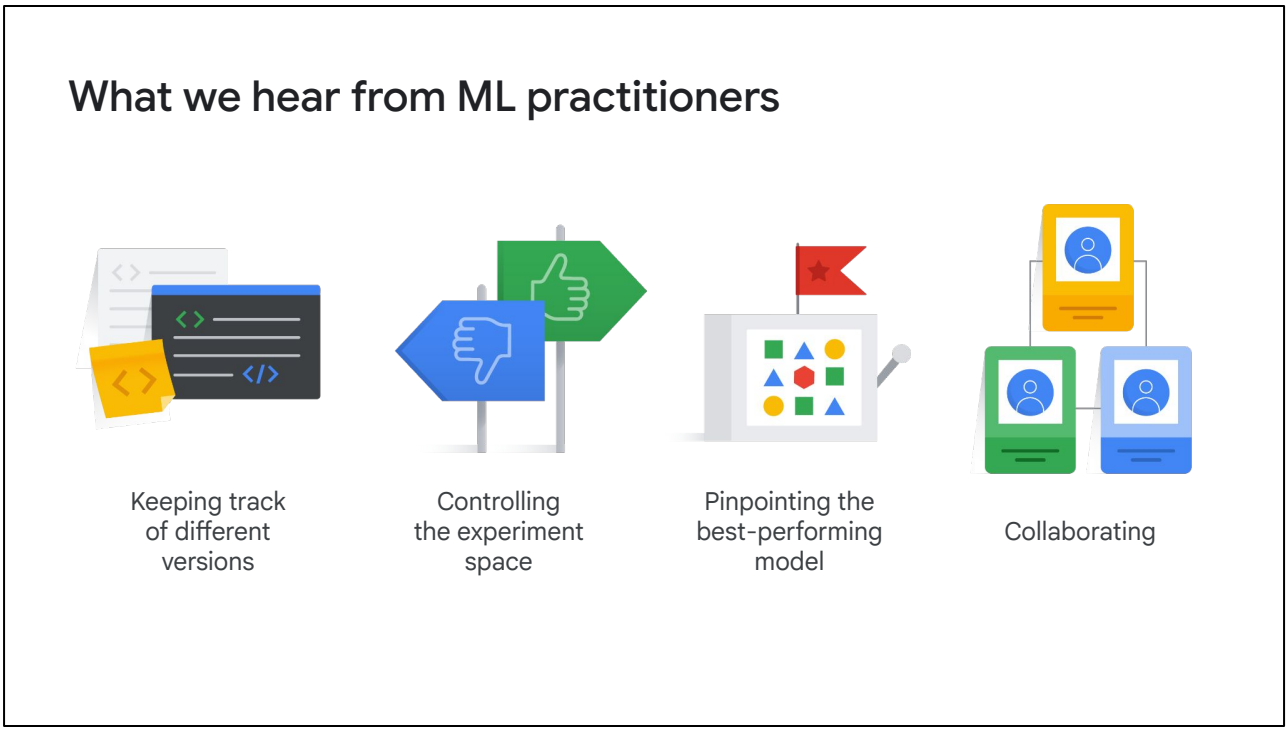
01 ML practitioners' pain points ☑

02 The concept of DevOps in ML ☐

03 The three phases of the ML lifecycle ☐

04 Automating the ML process ☐

Now, let's look at the challenges these ML practitioners face when they operationalize and make their models available for production.

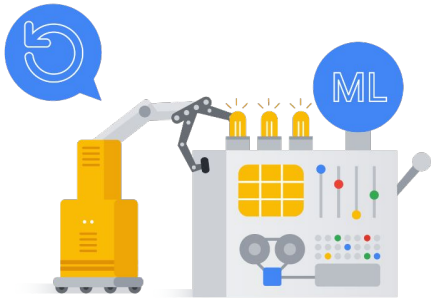


These challenges include managing and keeping track of complex details, such as data, model architectures, hyperparameters, and experiments.

As for specific pain points, we hear it can be challenging to:

- **Keep track of different versions** of the models and their codes, different training procedure parameters, hyperparameter values in each trial, and performance metrics.
- **Control the experiment space to advance**. In every iteration, these practitioners need to monitor what changes are being made, which ideas are being tried, which ideas work, and which ideas don't.
- **Pinpoint the best performing model** when the models are benchmarked against each other. The best model here refers to the one that delivers the ideal result for your specific use case.
- **Collaborate with data scientists, data engineers, ML engineers, application developers, site reliability engineers, business analysts, and business users** in operationalizing the ML models.

What we hear from ML practitioners



And then there is the matter of **reproducibility**.

Deploying a model to a production environment is difficult unless it can be reproduced. In fact, bypassing the reproducibility of the model is often discouraged or disallowed by policies or regulations.

Reproducibility can be a major concern for ML practitioners, because they want to be able to rerun the best model with a more comprehensive parameter sweep.

When a team successfully trains and makes a model ready for production in a streamlined fashion, performance and agility are considerably improved.

Even if there's a manual review step in the pipeline, automation ensures that each job is configured and executed in a repeatable manner, which reduces the risk of errors.

Also, for a production application, the model needs to be updated regularly as new data comes in.

Therefore, traceability becomes paramount.

01 ML practitioners' pain points

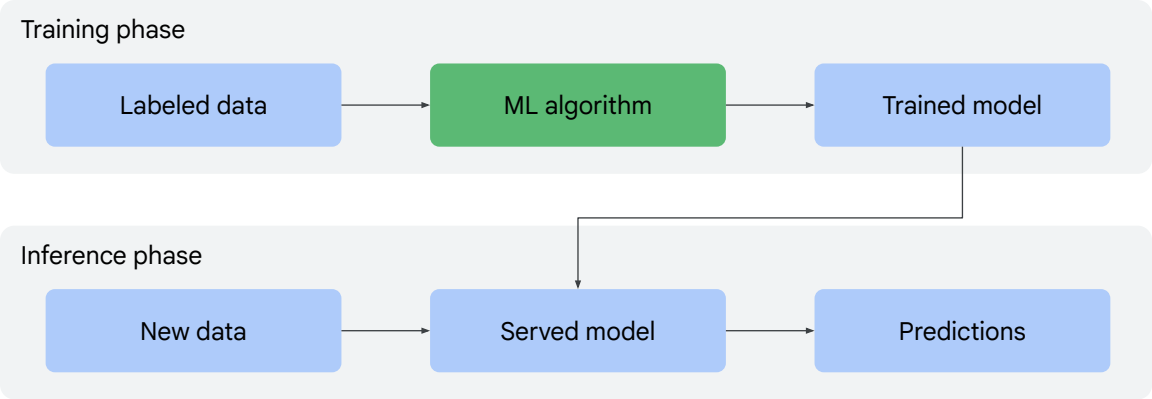
02 The concept of DevOps in ML

03 The three phases of the ML lifecycle

04 Automating the ML process

Now, let's walk through the phases of developing and publishing a machine learning model.

Operationalizing a model means balancing time, resources, and quality throughout the whole system



Given all the barriers that ML practitioners face today, what can be done to mitigate them?

One option is to look at machine learning projects through an operational lens.

Operationalizing a model means balancing time, resources, and quality throughout the whole system.

In software engineering, this approach is called DevOps.

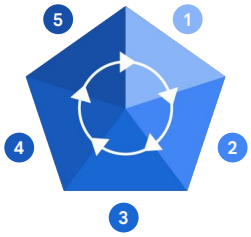
That term transfers over to machine learning as MLOps, which aims to streamline AI and ML projects in the same way that DevOps does for application projects.

MLOps is a lifecycle management discipline for machine learning.

Think of MLOps as a lifecycle management discipline for machine learning.

Its goal is a balanced process-driven approach to the management of resources, data, code, time, and quality to achieve business objectives and meet regulatory concerns.

Some DevOps concepts translate directly to MLOps



Continuous integration (CI)

1. Checkout the code.
2. Complete the task.
3. Validate against the code base.
4. Perform unit testing.
5. Merge the code.

There are many similarities between DevOps and MLOps.

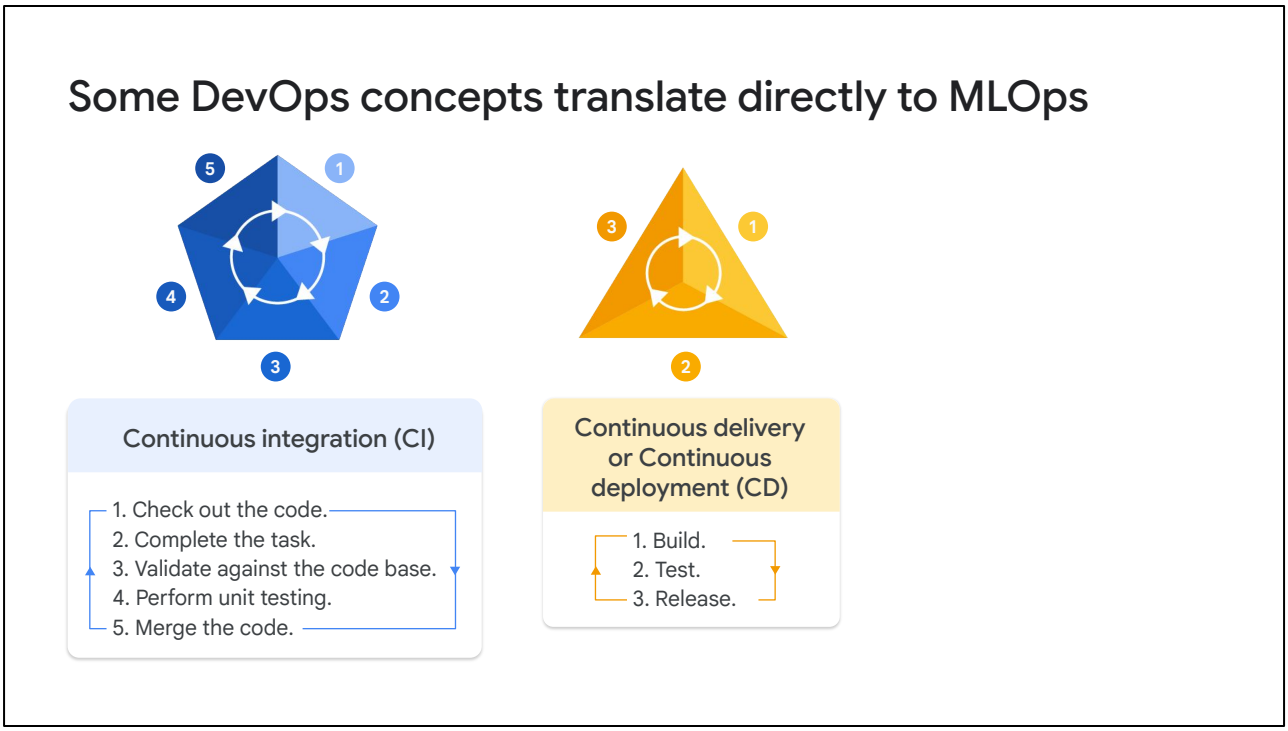
For example, when software developers work on a project, they don't all work on the same code at the same time. Instead, they work on a code repository and use source control and a branching strategy, which helps define how each feature, improvement, or bug fix of the product is handled. Code repositories and source control tools provide developers with both a historical record of all tasks, which lets them go back to previous versions, and a space to work on separate tasks in parallel where they can merge their efforts later. A branching strategy lets multiple teams work independently and verify that the changes on each branch do not conflict.

1. Whenever developers want to make changes to the source, they download a copy of the code from the code repository or "check out" the code in short.
2. The code files appear in their local directory and let developers use and compile the code. This way they can make changes to the source and to upload back to the repository later. In short, they complete the task on a branch in this phase.
3. Before the code is returned to the repository, developers need to check whether anything changed in the main version. In other words, developers need to ensure that their changes are made against the latest version of the codebase.
4. Before merging their changes, they also performs unit tests.
5. After developers confirm that their code behaves as they expected, they give permission to the code to be merged to the main branch.

When these changes are frequently merged with the main code, the chances of divergence decrease.

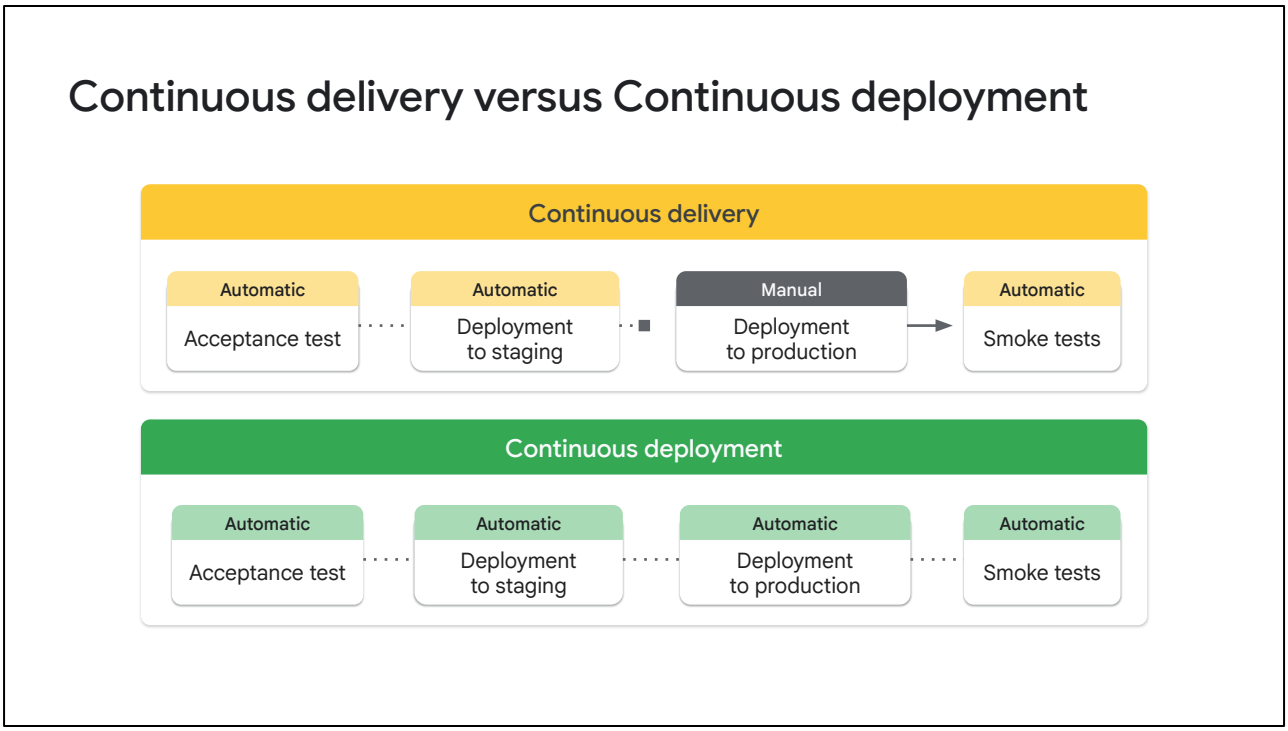
Branches inevitably diverge, and they diverge exponentially as they run without integrating. Therefore, how often you integrate your branches matters. Developers perform mainline integration when they have a healthy commit they can share, usually less than a day's work. For every push to the repository, it’s very useful to create a set of scripts to build and test your application automatically. These scripts help decrease the chances that you introduce errors in your application.

This process is called continuous integration, or CI for short. In a busy development team, this happens several times a day. Any change in the codebase repository, often triggers a CI pipeline. The pipeline is a series of steps that starts from checking out the code stage and ends with merging it.



Another process favored by developers is continuous delivery or continuous deployment which depend on the automation level.

An important caveat here is that continuous delivery and continuous deployment are not same, although they are both shortened as CD.



Both of them refer to a method for building, testing, and releasing software in short cycles.

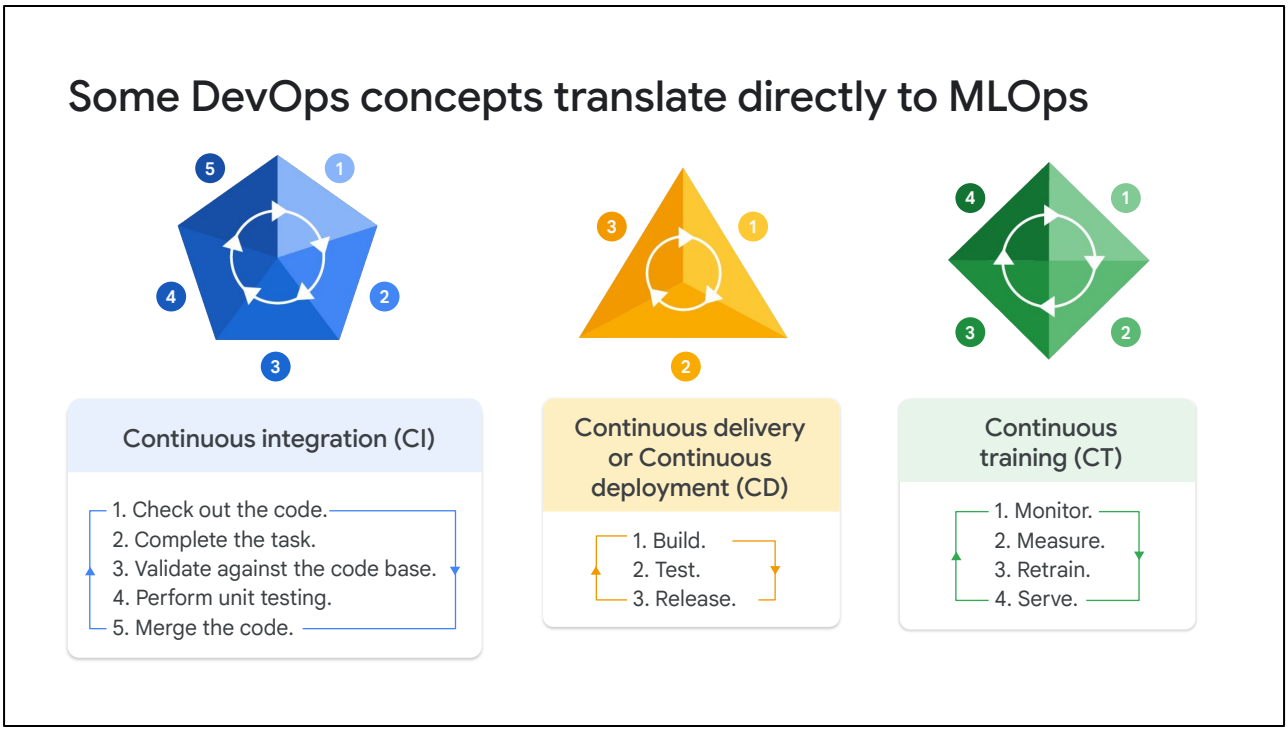
Done this way, the main development code is almost always production-ready and can be released into the live environment at any time.

If it is not done this way, the main code is like a race car with its wheels off and its engine out; it can go fast but only after it's put back together.

The difference between these two terms is the automation of deployment.

Continuous delivery automates integration or acceptance tests, deployment to staging, and smoke tests. Note that staging refers to a production-like environment and smoke tests refer to testing the application before it is deployed. In continuous delivery, deployment to the production is still done manually.

However, **continuous deployment** compliments continuous integration with additional steps by automating the configuration and deployment of the application to the production environment.



Along with continuous integration, delivery and deployment, *data* is also an important aspect of MLOps.

Unlike conventional software, which performs the same action repeatedly, an ML model can go “off.” This means that its predictive power can wane as data profiles inevitably change. A model deployed in production will not be able to adapt to changes in data by itself.

This requires something called continuous training, or CT for short.

Continuous training is the process of monitoring, measuring, retraining, and serving ML models automatically and continuously to adapt to changes in the data before they’re redeployed.

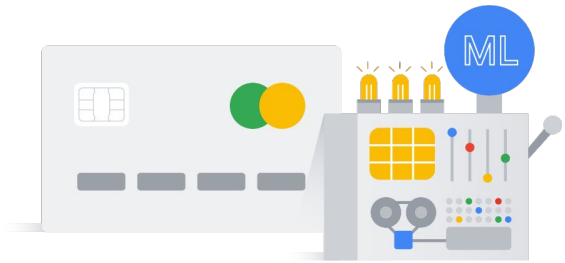
But MLOps differs from DevOps in important ways

	DevOps	MLOps
01	Tests and validates code and components.	Also tests and validates data, data schemas, and models.
02	Focuses on a single software package or service.	Also considers the whole system: the ML training pipeline
03	Deploys code and moves to the next task.	Constantly monitors, retrains, and serves the model.

MLOps differs from DevOps in important ways, too:

- Continuous integration is no longer only about testing and validating code and components, but also about testing and validating data, data schemas, and models.
- It's about a whole system—the ML training pipeline—that automatically deploys another service: the model prediction service.
- Uniquely, ML is also concerned with automatically monitoring, retraining, and serving the models.

Machine learning is the high-interest credit card of technical debt



Credit card bill

This is a bill you have to pay. If you do not pay within one month, a 30-day surcharge is assessed.

Name: A. Developer
Address: 29 Dev Team Street
Bill received: July 20, 2020

Your transactions

Item	Price
Not enough time on data selection	30 days
Insufficient feature engineering	45 days
Batch processing of live data	30 days
Different data in live model	90 days
Subtotal	195 days
Penalties and interest	60 days
Total to pay	255 days

Another concept that transfers well from software development to machine learning is *technical debt*.

Technical debt represents the pressure to prioritize releases over quality, which might mean not paying close attention to code quality.

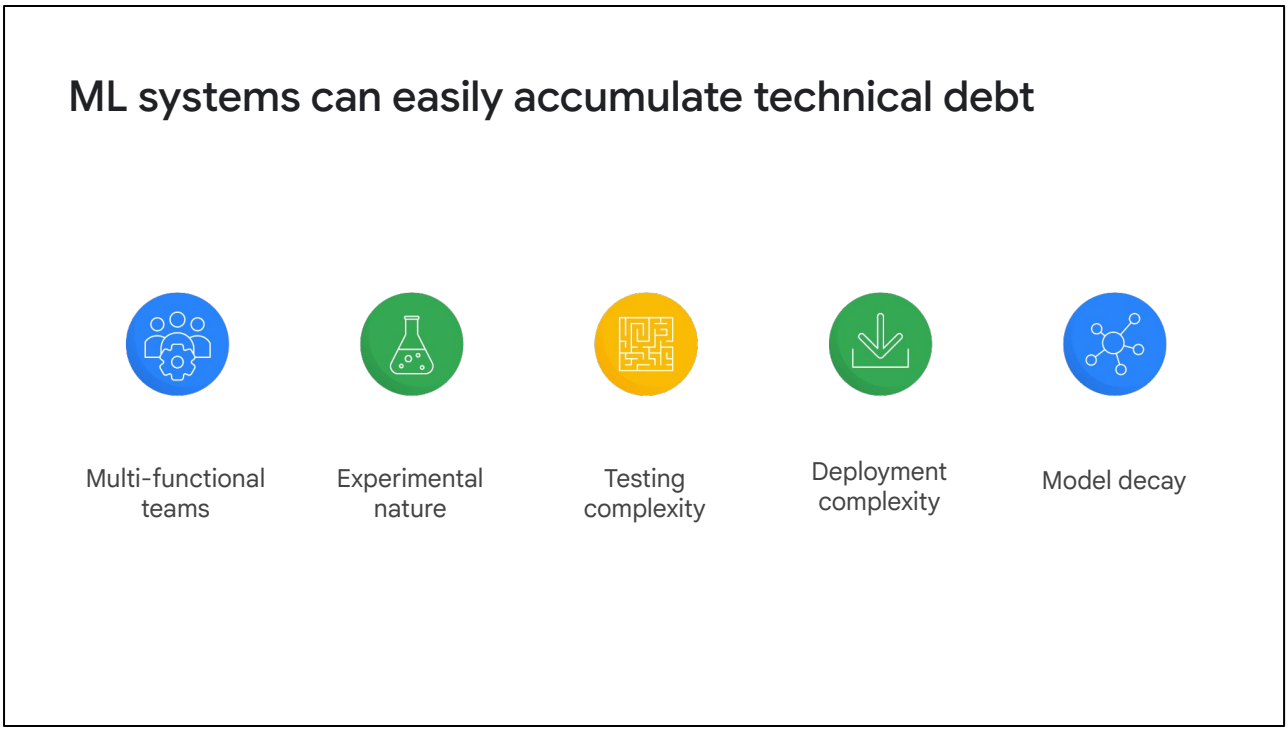
There's often a compromise between faster delivery and high quality. Although there might be good reasons to prioritize the release, developers will still need to go back and fix things later.

Machine learning could arguably be considered "the high-interest credit card of technical debt." This means that developing and deploying an ML system can be relatively fast and cheap, but maintaining it over time can be difficult and expensive.

The real challenge isn't building an ML model; the challenge is building an integrated ML system and operating it in production continuously.

Just like a high-interest credit card, the technical debt with machine learning compounds and can be incredibly expensive and difficult to pay down.

Please check the reading at the end of this section to learn more about The High Interest Credit Card of Technical Debt.



Machine learning systems have all the operational challenges of software development, and a few of their own.

- One challenge is **multi-functional team** structure. Unlike many business projects, which are usually managed in a single department, ML projects require significant multi-functional expertise.

- Another challenge stems from the fact that **ML is experimental in nature**, which requires constantly approaching the data, the models, and parameter configuration in new ways. The real challenge comes when ML practitioners try to track all experiments and metadata while they try to maintain reproducibility and maximize code reusability.

- And then there is **testing ML systems**, which is more complex than testing other software systems because you need to validate data, parameters, code, and hyperparameters together in a system, instead of only unit-testing methods or functions.

- In ML systems, deployment isn't as simple as deploying an offline-trained ML model as a prediction service. ML systems might require you to **deploy a multi-step pipeline** to automatically retrain and deploy models.

- And finally, ML systems can present challenges related to concept drift and model decay. Data profiles constantly change. If something changes in the

- data input, the predictive power of the model in production will probably change with it. Therefore, you need to track summary statistics of your data and monitor the online performance of your model to send notifications or roll back when values deviate from your expectations.

  Drift is the change in an entity regarding a baseline. With a production ML model, this is the change between the real-time production data and a baseline dataset, most likely the training set, that's representative of the task the model is intended to perform. To learn more about the several types of drift in ML models, check the Production Machine Learning Systems course in the Google Cloud catalog.

Technical debt accumulates in an ML system for many reasons, so we look at ways to mitigate that throughout this course.

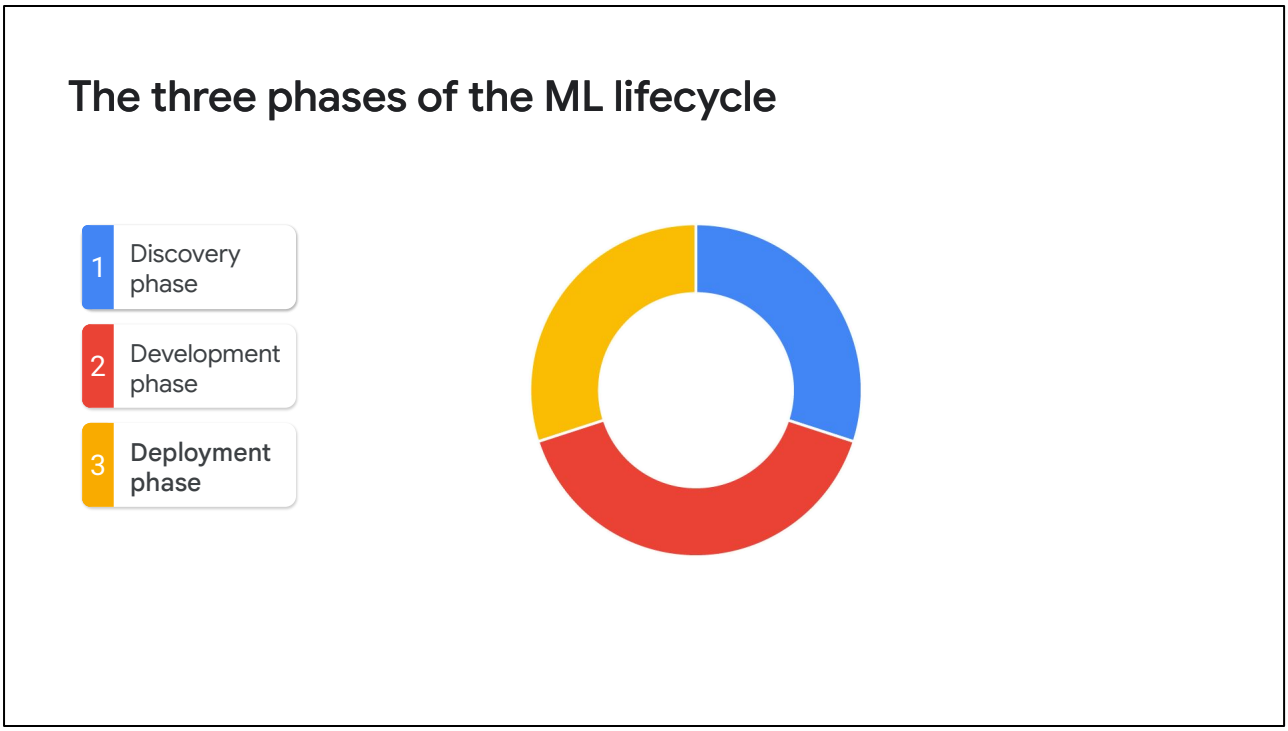
01 ML practitioners' pain points

02 The concept of DevOps in ML

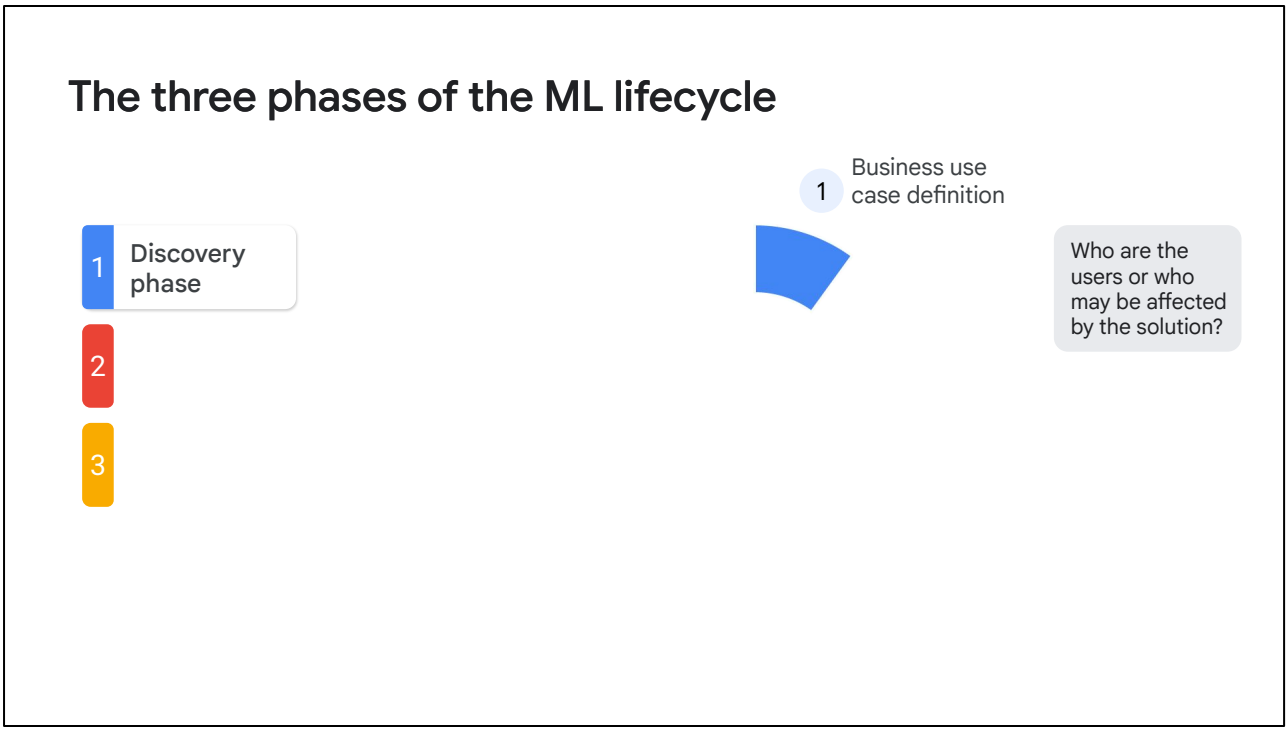
03 The three phases of the ML lifecycle

04 Automating the ML process

Let's explore the three main phases of a machine learning lifecycle and see how they map to tasks within MLOps.



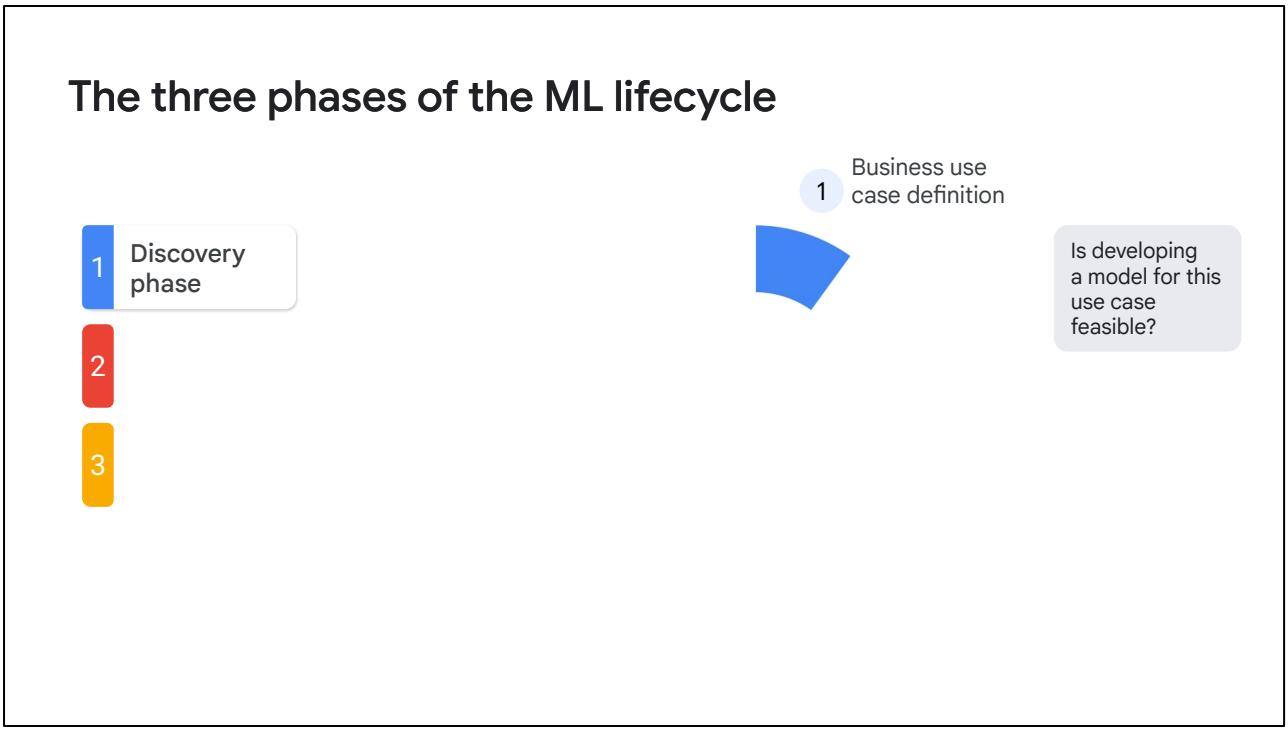
- The first is the discovery phase,
- the second one is the development phase,
- and the third one is the deployment phase.



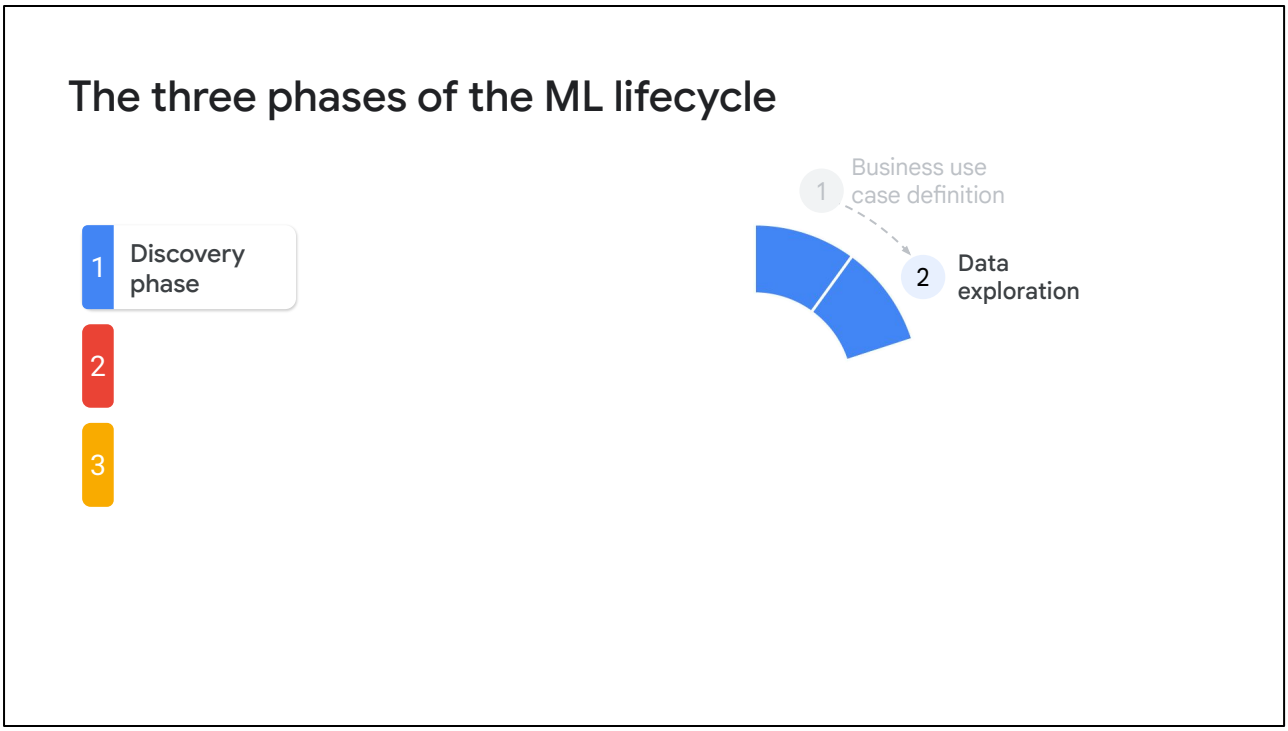
The **discovery phase** is where the problem is defined and the desired outcome is identified.

Some of the tasks performed in this phase can be the following:

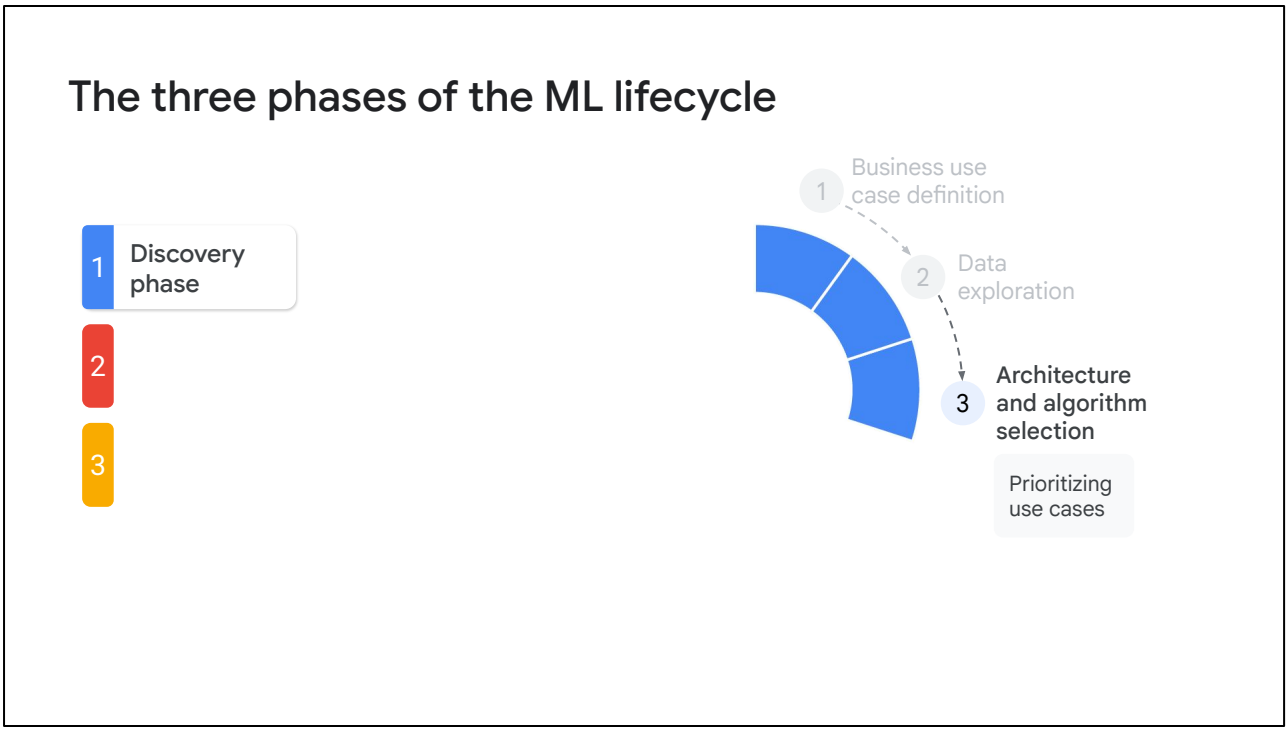
- **Business use case definition:** Identifying the business need and the desired outcome allows for a clear plan of what a machine learning model will help achieve.
- **Contextual understanding**: The discovery phase also includes gaining contextual understanding of the people who are using or may be affected by the solution. This critical information can help further define the problem or task that needs to be solved.



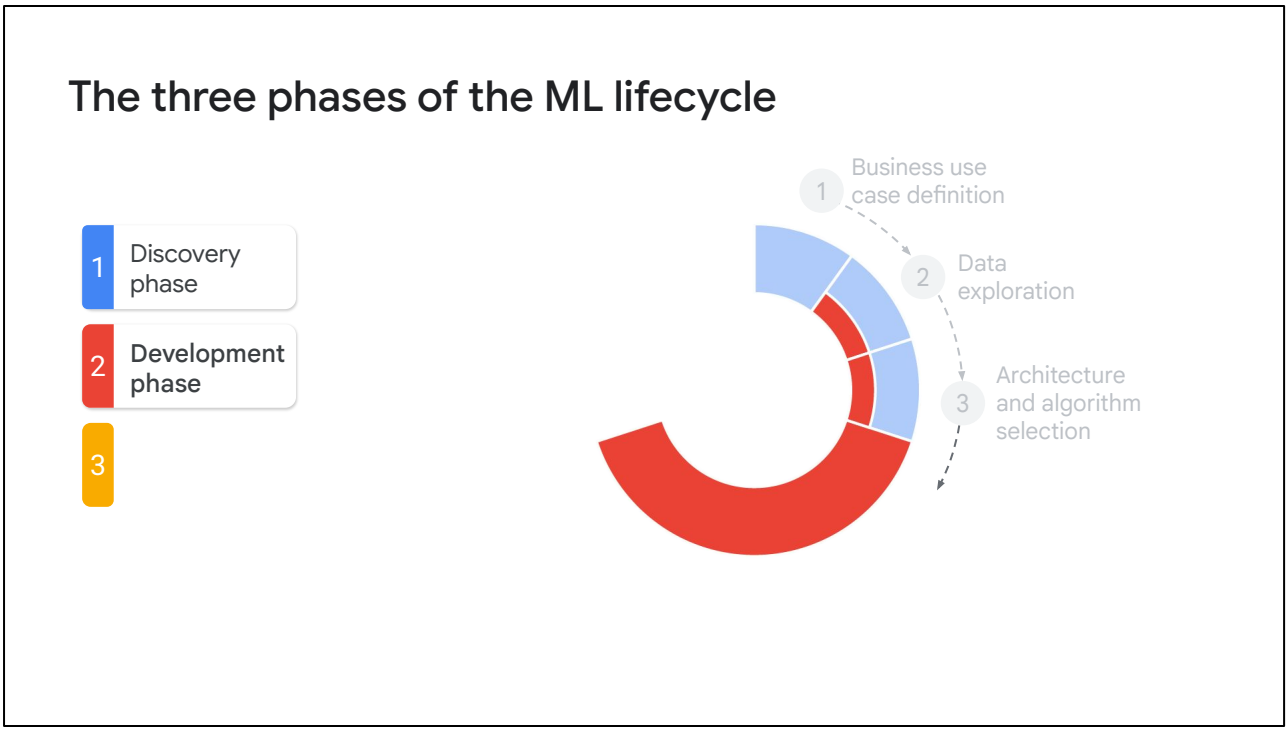
- **Use case feasibility:** It's also important to assess whether the problem that you're trying to solve can actually be solved with machine learning. Let's say that you have historical data that can't be accessed during inference time. If that's the case, ML might not be a feasible option to solve your problem. You'll probably need to perform a more comprehensive analysis before moving forward.



- **Data accessibility and data exploration**: During this phase, it's important to determine which datasets are needed for the ML model, and whether that data is readily available and good enough to train the model. And if external datasets are needed, you must decide how to acquire them.



- **Architecture and algorithm selection:** Then, depending on the task that will be performed, the team chooses an architecture and a machine learning algorithm.
- **Prioritizing use cases:** Another aspect of the discovery phase is prioritizing the different use cases that the business has as potential ML projects, but this discussion is out of scope for this particular course. Check the [Managing Machine Learning Projects with Google Cloud course]() to learn more about identifying business value for using ML.



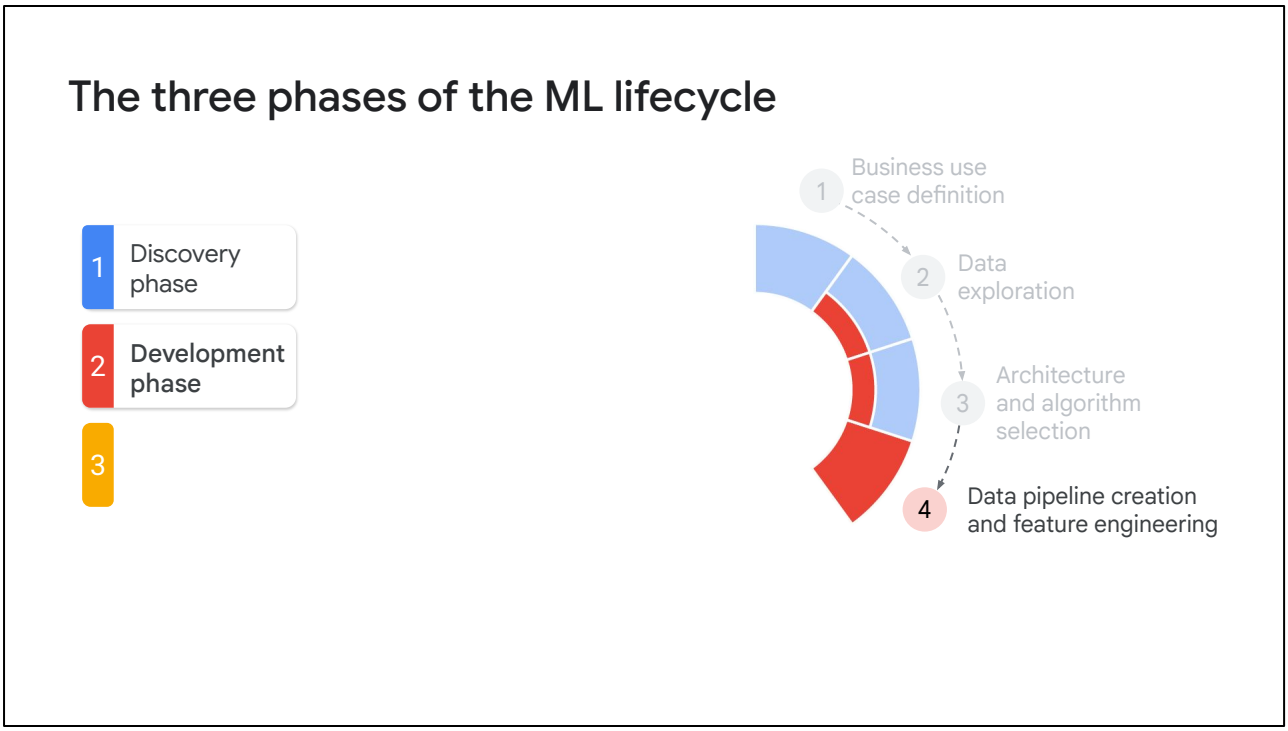
The development phase: the second phase of the Machine Learning lifecycle is the **development phase**.

After the feasibility assessment is complete and you've got the *go ahead*, development can start.

However, sometimes you might see that the development phase starts during data exploration, which is part of the discovery phase.

Then you might ask this question: shouldn't we wait until the results of the feasibility study are available?

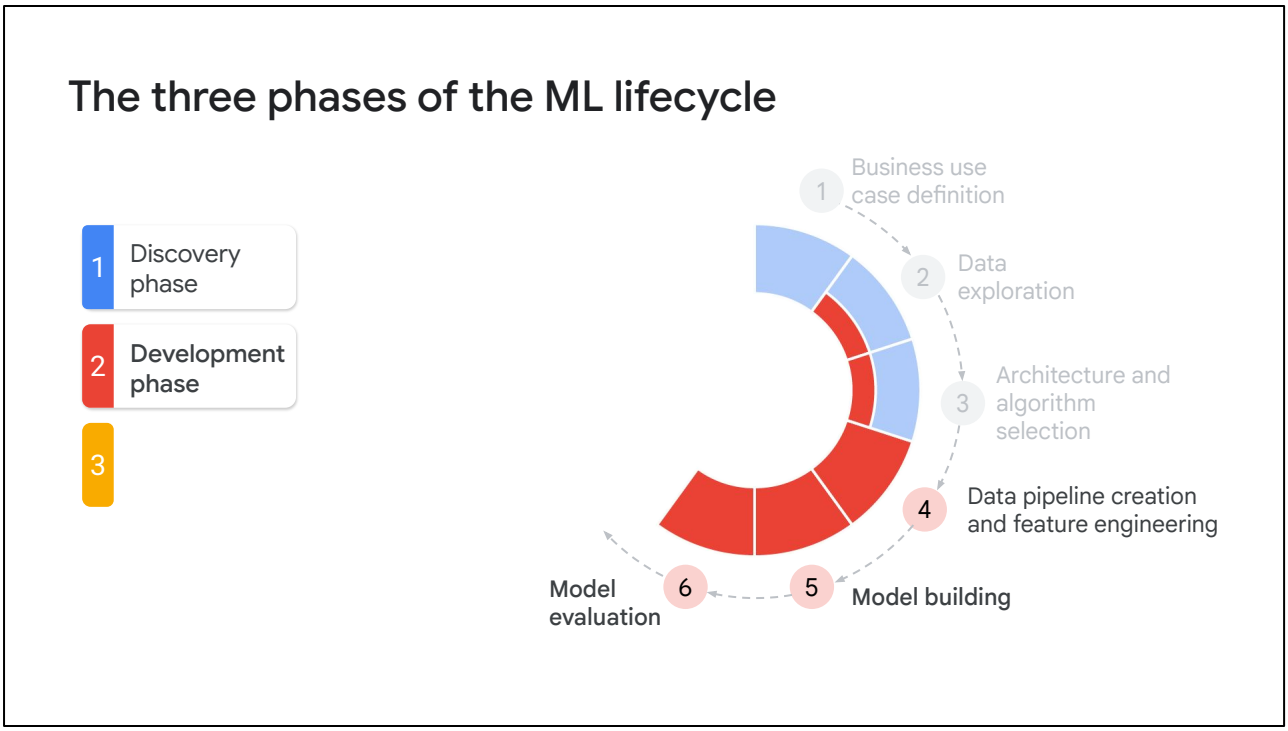
However, in reality, even for data exploration and algorithm selection, some proof of concepts might need to be developed, and that is why the development phase and discovery phase overlap during the data exploration.



It starts with a **data pipeline creation and feature engineering**. This is when all of the necessary data is cleaned, extracted, analyzed, and transformed for us in the model.

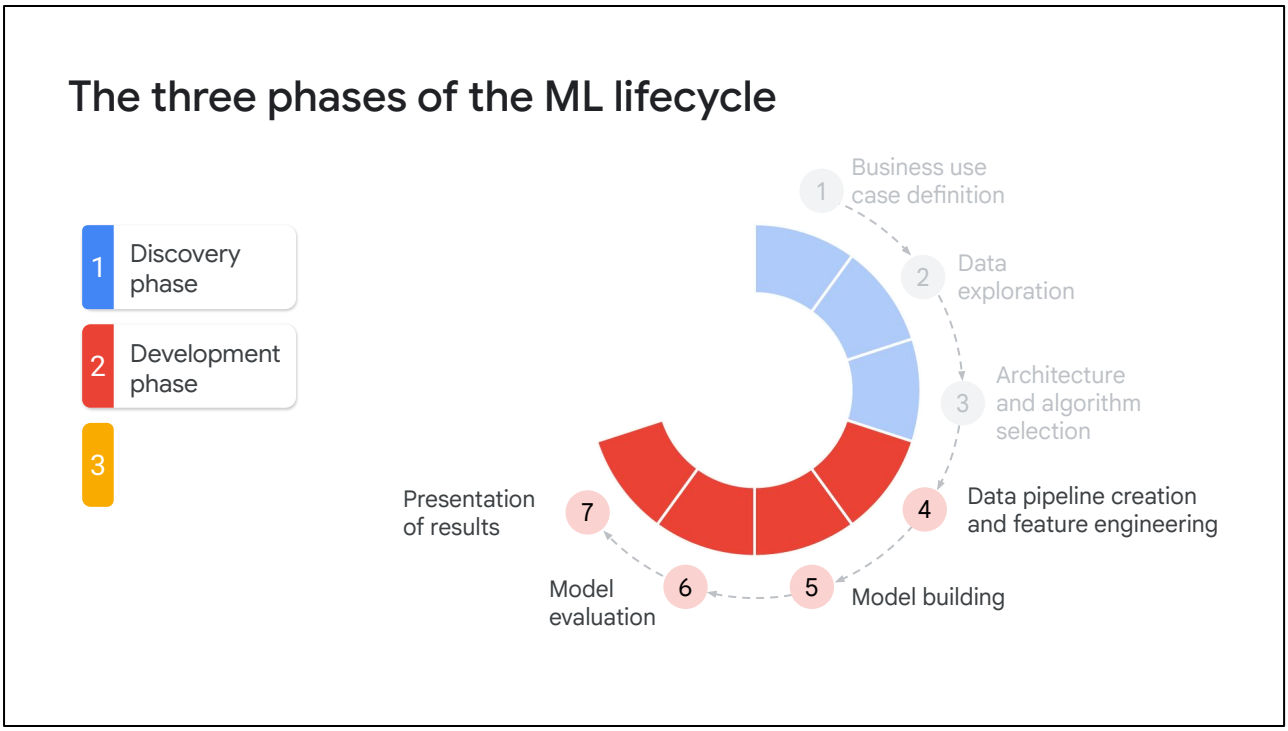
A data pipeline helps ensure that all data operations will perform as expected–for both offline and stream data–and avoid data skew.

You can visit the [data preprocessing for ML documentation]() to learn more about best practices for preprocessing data in an ML pipeline.

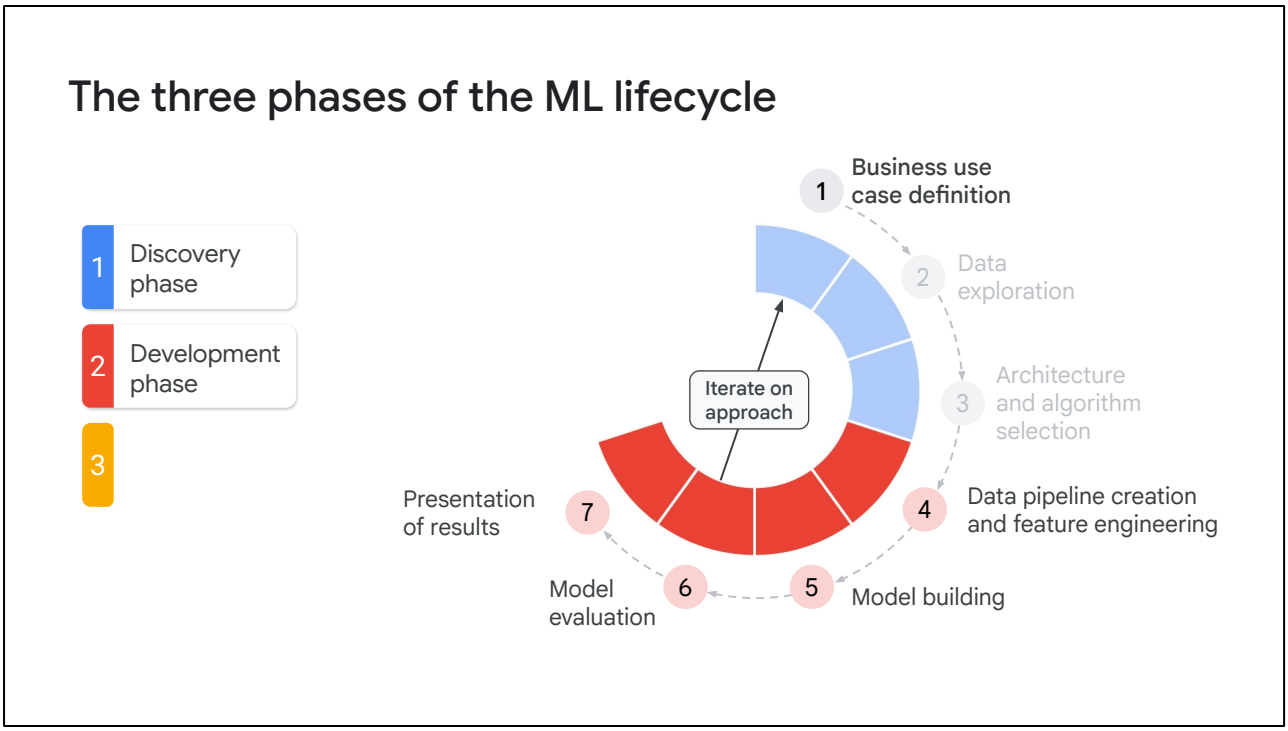


After the data is ready, model building and the model evaluation begins.

The word "begins" is used here because these steps might require a couple of iterations until the ML practitioners are happy with



the results and are ready to present them to the main stakeholders.

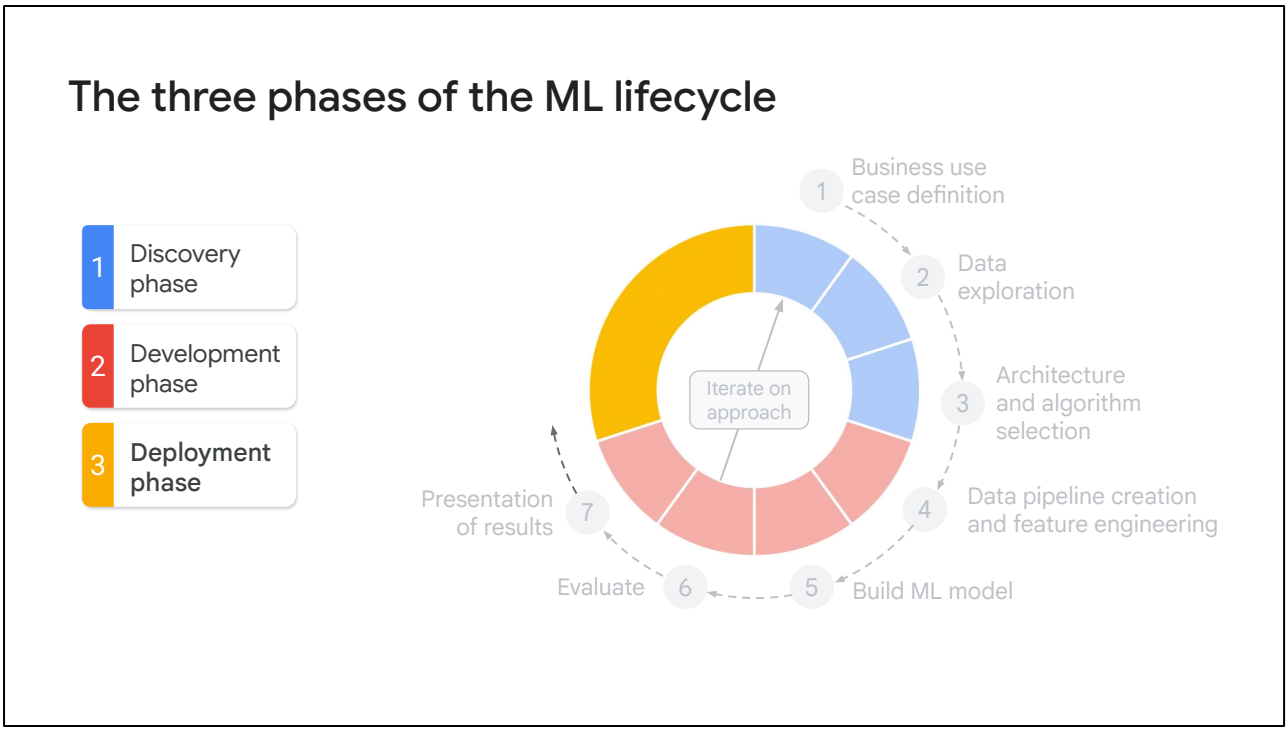


Iteration is a central concept of machine learning.

Constant iteration, tuning, and improvement is often required to ensure that a model performs as planned.

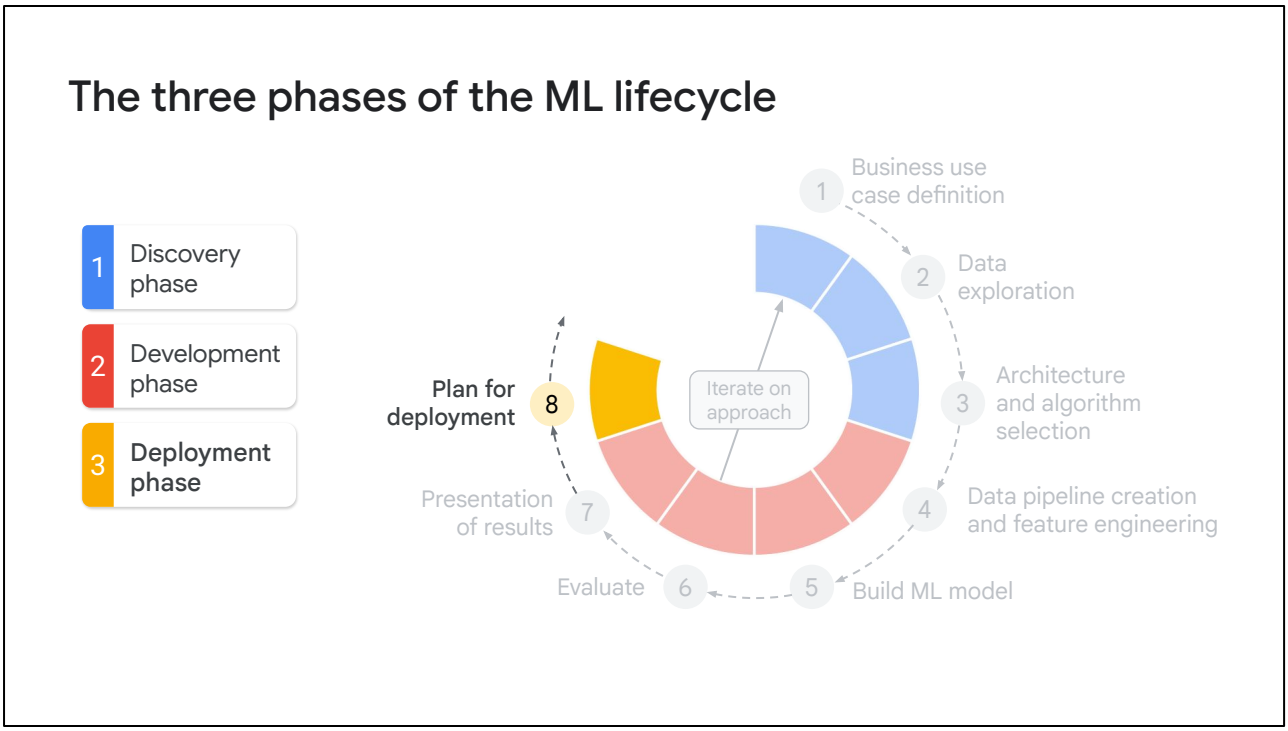
During this phase, it might be helpful to perform some, or all, of the following activities:

- First, revisit the model use case. Because the ML algorithm might not identify data patterns for your use case in the first iteration, you might want to ensure that the model will still help you achieve your goal.
- Revisit the chosen dataset to determine whether the model requires a higher volume or new features. Data should be revisited because the model either needs more of it or it needs additional aspects or new features from the existing data.
- And finally, it might be helpful to consider whether additional data transformations are needed to improve the model quality, or whether a different algorithm might be better.



The deployment phase: The last phase of the ML lifecycle is the **deployment phase**.

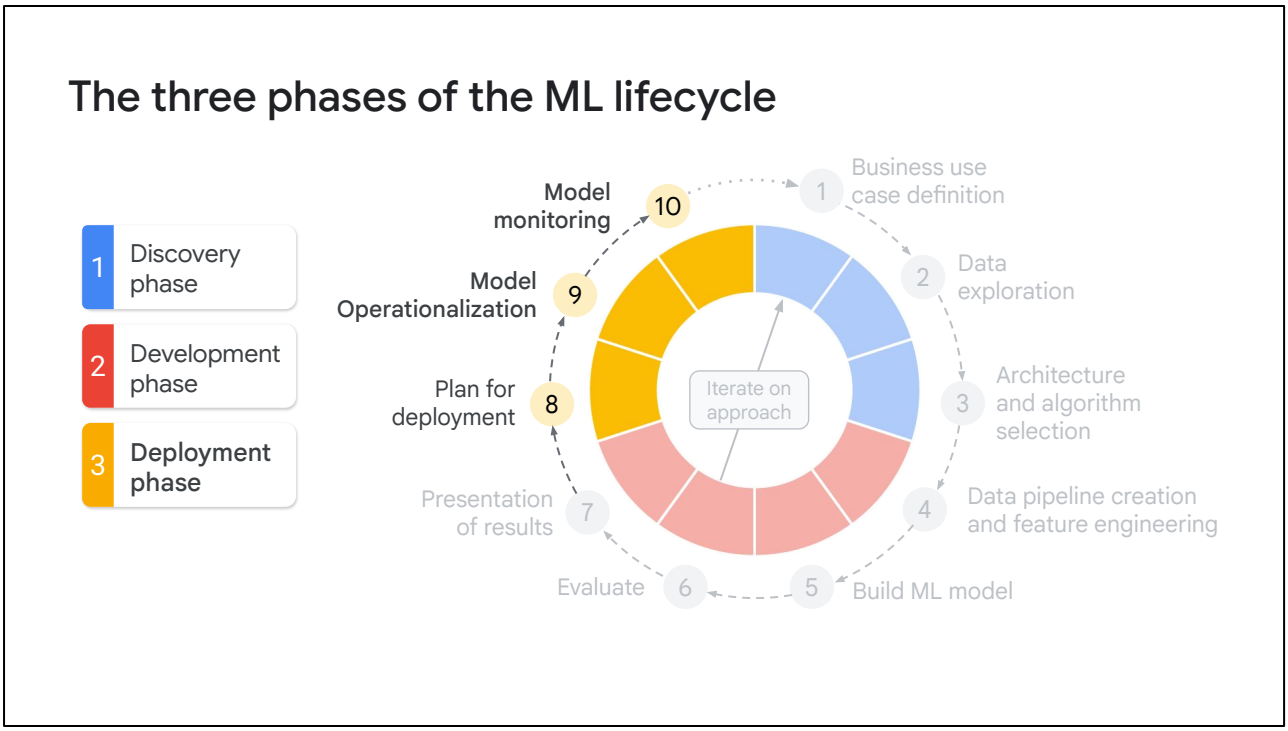
Some of the tasks performed in this phase can be the following:



- **Plan for deployment:** After results are presented to the stakeholders and all parties are satisfied with the model performance, it's time to plan for model deployment.

  This is when the following questions can arise:

  - Which platform should host the model?
  - Which ML model serving tools are needed?
  - And how many nodes are needed for the cluster to scale the model in a cost-effective manner?



- **Model Operationalization and Monitoring:** As previously mentioned, operationalizing and monitoring a model helps with maintenance and avoiding model decay.

  Implementing a strategy to detect a concept or data drift allows signaling when the model needs to be retrained or when the data needs to be adjusted or augmented.

  Many people could be reliant on your model's predictions, so it's important to ensure that your pipeline accounts for all the necessary health checks and alerts.

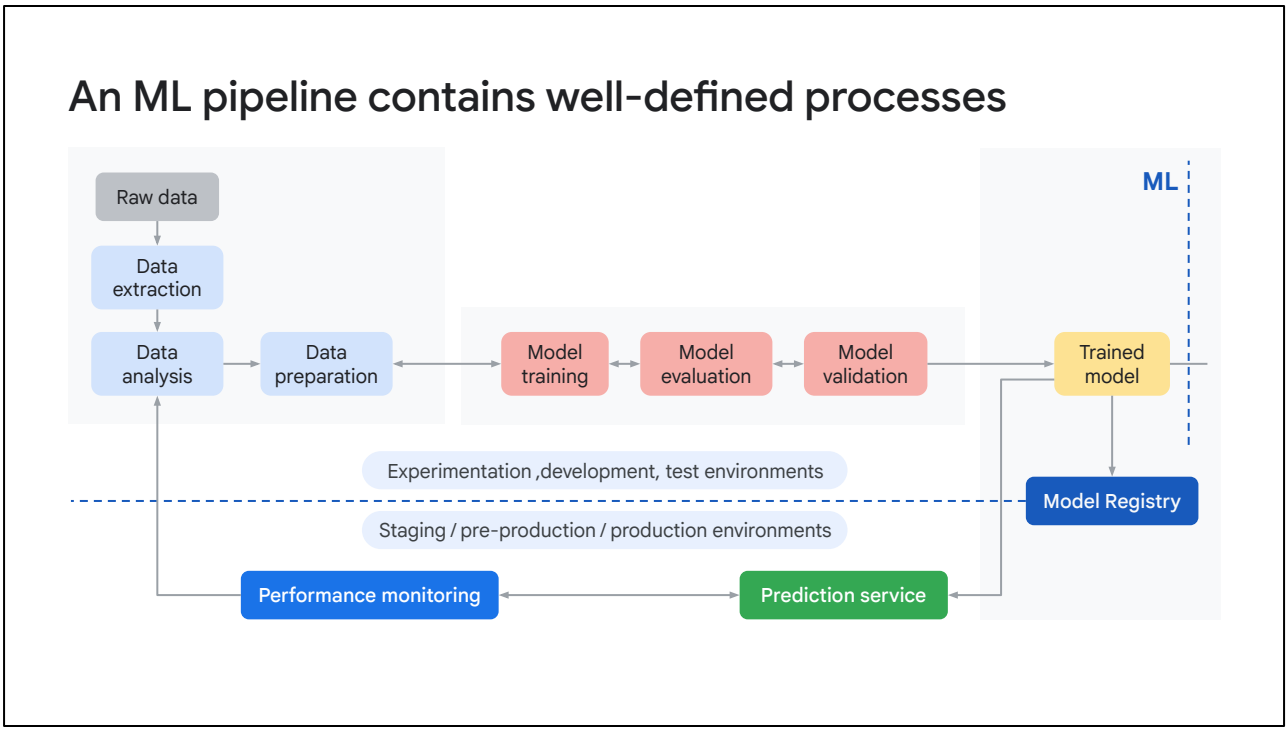
01 ML practitioners' pain points ☐

02 The concept of DevOps in ML ☐

03 The three phases of the ML lifecycle ☐

04 Automating the ML process ☑

In any ML project,



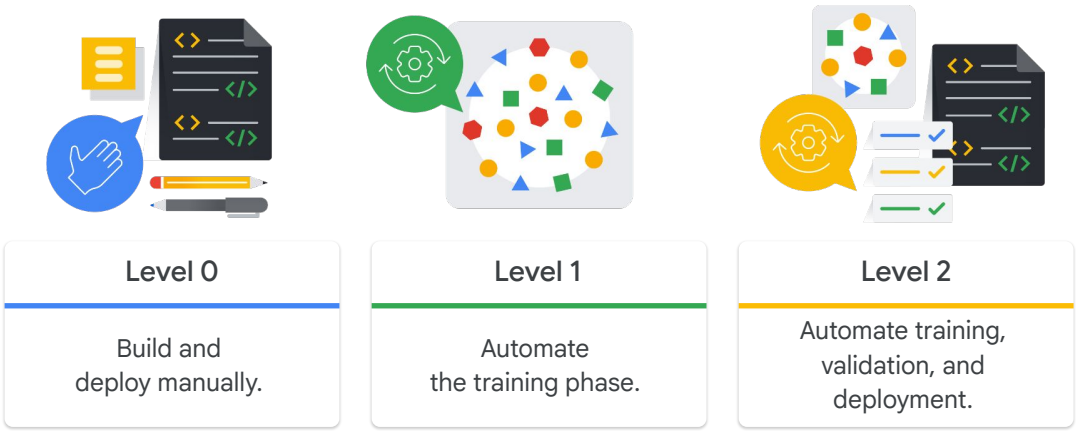
after you define the business use case and establish the success criteria, the process of delivering an ML model to production involves many steps.

These include:

- Data extraction
- Data analysis
- Data preparation
- Model training
- Model evaluation
- Model validation
- Model serving
- And model monitoring

These steps can be completed manually or by an automatic pipeline.

The level of automation defines the maturity of the ML process



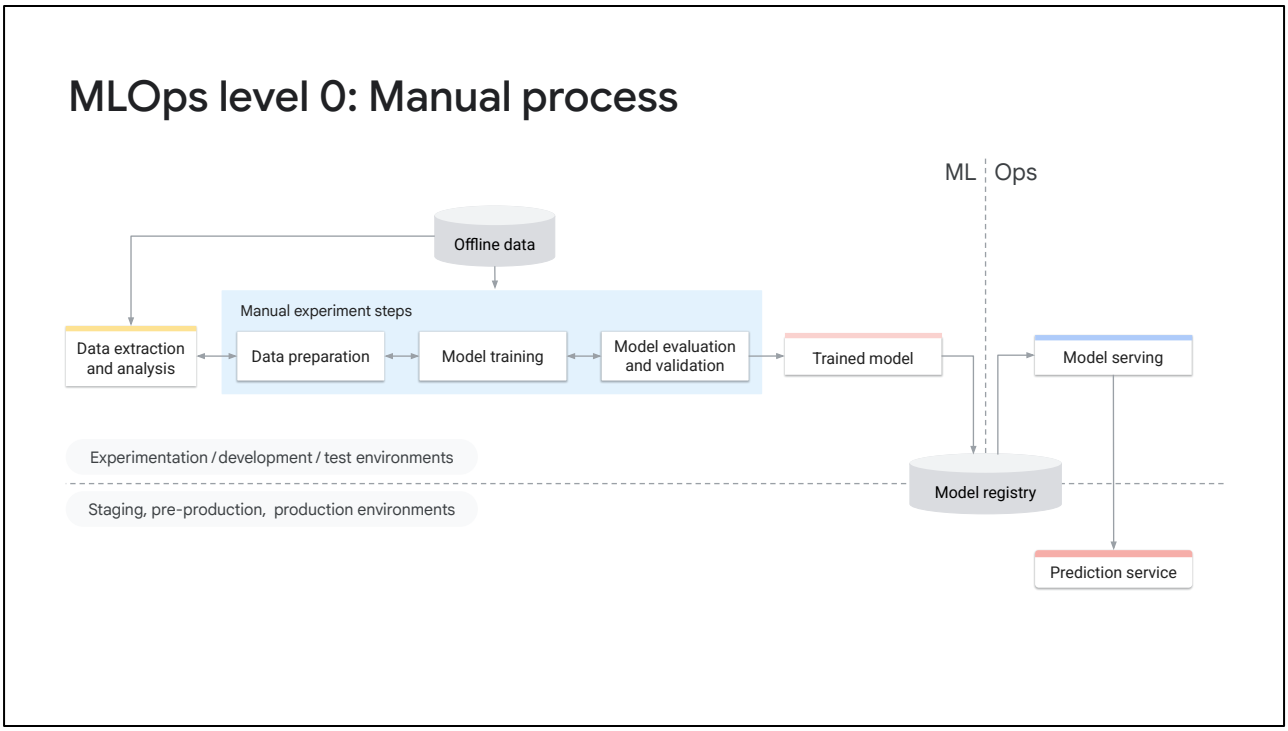
The level of automation of these steps defines the maturity of the ML process.

Many ML professionals build and deploy their models manually. This is called maturity level 0.

Other ML practitioners perform continuous training of their models by automating the ML pipeline. This is *maturity level 1*.

Finally, the most mature approach completely automates and integrates the ML training, validation, and deployment phases. This is *maturity level 2*.

Let's look at each of these levels of automation in a little more detail.

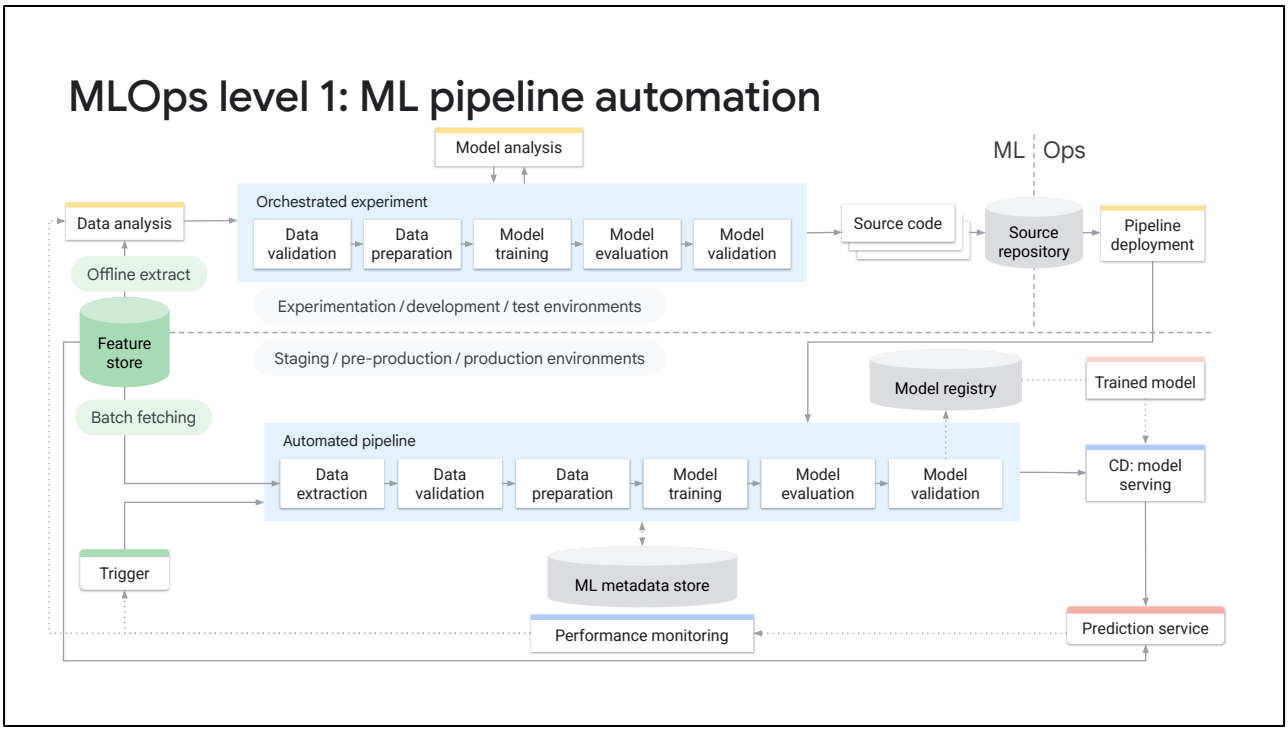


At the basic level of maturity, or level 0, many teams have data scientists and ML researchers who can build state-of-the-art models, but their process for building and deploying ML models is entirely manual.

The characteristics of MLOps level 0 are:

- A manual, script-driven, and interactive process.
- A disconnection between ML and operation teams.
- Infrequent release iterations.
- No continuous integration, continuous delivery or continuous deployment.
- Deployment refers to the prediction service.
- And a lack of active performance monitoring.

Check the documentation to learn more about the characteristics and challenges of MLOps level 0.



The goal of MLOps level 1 is to perform continuous training of the model by automating the ML pipeline. This lets you achieve the continuous delivery of the model prediction service.

To automate the process of using new data to retrain models in production, you must introduce automated data and model validation steps to the pipeline, as well as pipeline triggers and metadata management.

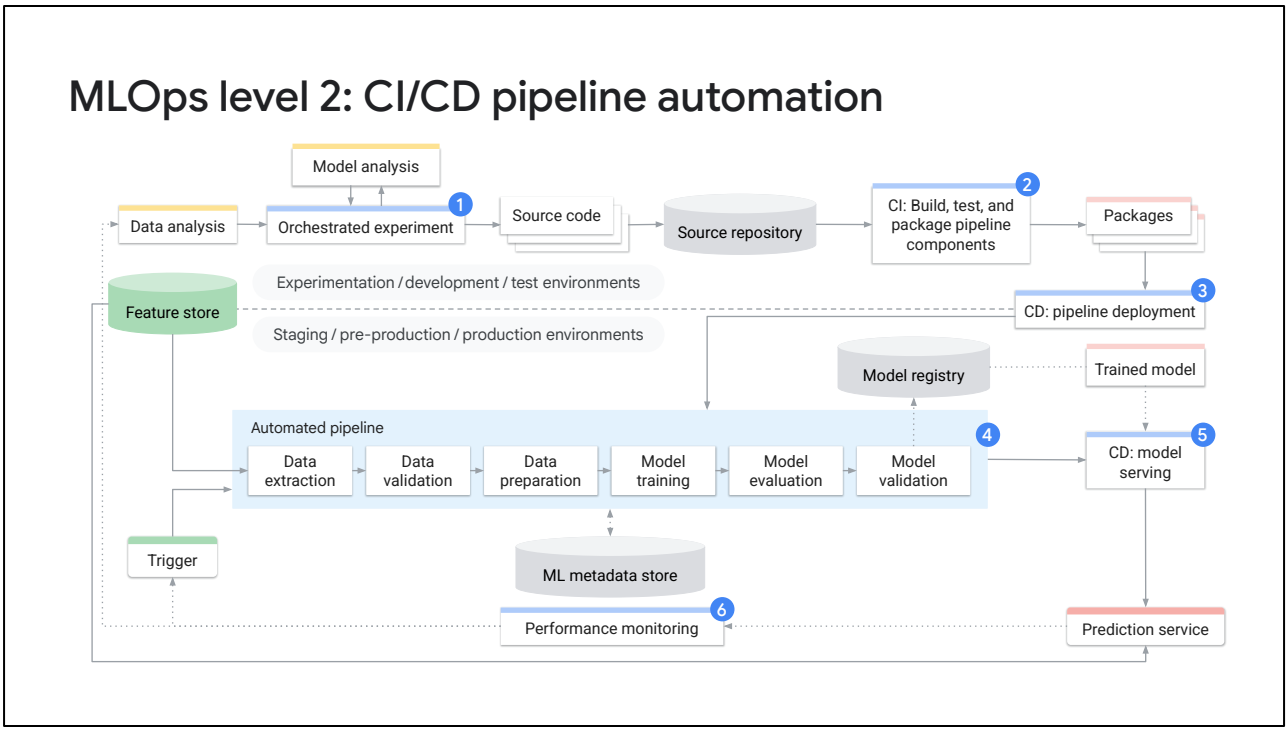
The characteristics of MLOps level 1 are:

- Rapid experiment
- Continuous training of the model in production
- Experimental-operational symmetry
- Modularized code for components and pipelines
- Continuous deployment of models, which means continuous delivery of models and automated pipeline deployment.

To learn more about the characteristics and challenges of MLOps level 1, check the documentation at the end of this section.

One important note is that to enable ML continuous training, you must to add a couple of the components, such as data and model validation, feature store, metadata management, and ML pipeline triggers to the architecture.

You'll explore all these components in upcoming courses of this series.

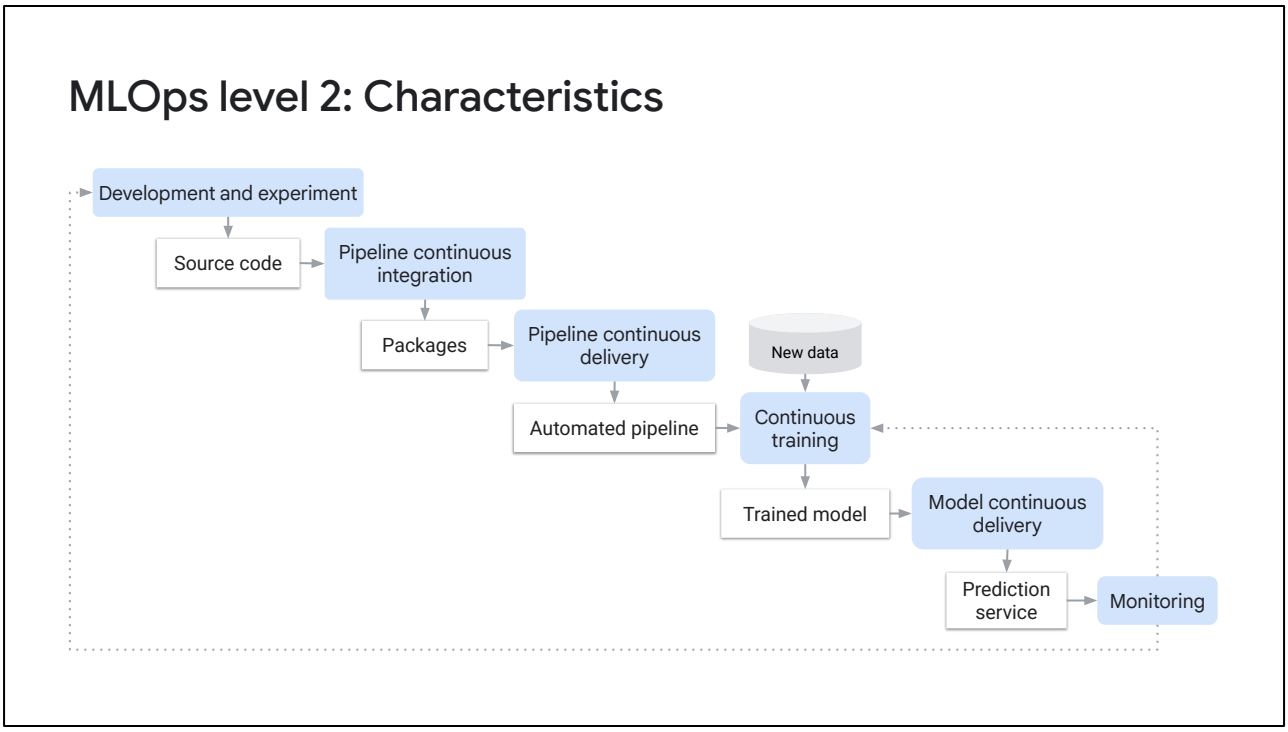


MLOps level 2 can be characterized as Continuous Integration/Continuous Deployment, or CI/CD, pipeline automation, because you need a robust automated CI/CD system for a rapid and reliable update of the pipelines in production.

This automated CI/CD system lets ML practitioners rapidly explore new ideas around feature engineering, model architecture, and hyperparameters.

MLOPs level 2 achieves this automation thanks to the following components:

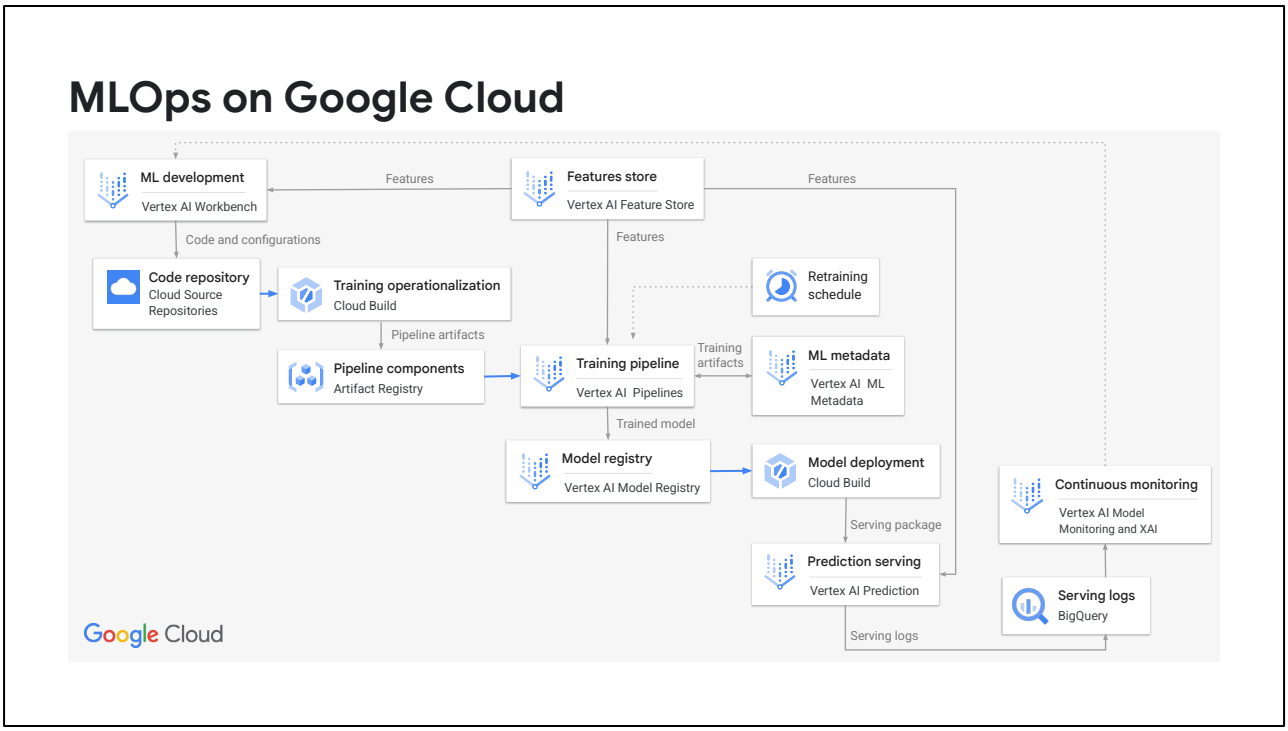
- Source control
- Test and build services
- Deployment services
- Model registry
- Feature store
- ML metadata store
- ML pipeline orchestrator.



In any ML project, after you define the business use case and establish the success criteria, the process of delivering an ML model to production involves many steps. These include:

1. Development and experimentation: you iteratively test new ML algorithms and new modeling where the experiment steps are orchestrated. The output of this stage is the source code of the ML pipeline steps that are then pushed to a source repository.
2. Then, pipeline continuous integration: you build source code and run various tests. The outputs of this stage are pipeline components such as packages, executables, and artifacts that are deployed in a later stage.
3. Next is pipeline continuous delivery: you deploy the artifacts produced by the CI stage to the target environment. The output of this stage is a deployed pipeline with the new implementation of the model.
4. Then there is automated triggering: the pipeline is automatically executed in production based on a schedule or in response to a trigger. The output of this stage is a trained model that is pushed to the model registry.
5. The next is model continuous delivery: you serve the trained model as a prediction service for the predictions. The output of this stage is a deployed model prediction service.
6. And finally, monitoring: you collect statistics on the model performance based on live data. The output of this stage is a trigger to execute the pipeline or to execute a new experiment cycle.

These steps can be completed manually or by an automatic pipeline.



In the full stack for a machine learning system with Vertex AI, you can use:

- Vertex AI Feature Store for feature store
- Vertex AI Workbench for model development
- Cloud Source Repositories as your code repository
- Cloud Build for training operationalization
- Artifact Registry for your pipeline components
- Vertex AI Pipelines for your training pipeline
- Vertex AI Model Registry as your model registry
- Vertex AI ML Metadata for your ML metadata
- Cloud Build for model deployment
- Vertex AI Prediction for prediction serving
- BigQuery for serving logs
- Vertex AI Model Monitoring and Vertex Explainable AI, or XAI in short, for continuous monitoring.

To learn more about the characteristics of MLOps level 2 and continuous integration/continuous delivery and continuous deployment concepts, check the documentation at the end of the course.